INFORME DE SEGURIDAD - SESION 10

Pruebas de seguridad y gestion de contrasenas en PharmaBackend

Estudiante	Josias Pari Mamani
Curso	Lenguaje de Programacion II
Rama	feature/seguridad-pruebas-pari
Proyecto	PharmaBackend

C1. Resumen de pruebas ejecutadas

Se ejecutaron 28 casos documentados: 16 casos base, 4 casos propios de grupos distintos y 8 casos adicionales de las partes B1, B2 y B3. En la ejecucion final todos los casos obtuvieron el resultado esperado; por ello no se registraron fallos funcionales de seguridad en la matriz final.

Grupo	Ejecutados	Pasan	Fallan
Autenticacion	5	5	0
Token JWT	6	6	0
Permisos	5	5	0
Usuarios	3	3	0
CORS	1	1	0
B1 - Contrasena	4	4	0
B2 - Roles	2	2	0
B3 - Estado	2	2	0

C2. Fichas de hallazgos

Resultado: no se generaron fichas de hallazgos porque ningun caso de la ejecucion final quedo en estado Falla. Las incidencias operativas observadas durante la preparacion de las pruebas -por ejemplo, tokens vencidos durante una sesion o solicitudes no guardadas con el metodo correcto en Postman- fueron corregidas antes de registrar la evidencia y no representan defectos de seguridad del backend.

Evidencias utilizadas: capturas de Postman, Network del navegador para CORS y consultas SQL de usuarios/roles. La matriz completa se conserva en docs/matriz-seguridad-s10.md.

C3. Analisis de seguridad

1. Mensaje generico en el login

El login responde el mismo mensaje para un usuario inexistente, una contrasena incorrecta y una cuenta deshabilitada porque revelar la causa exacta permitiria enumerar cuentas validas. Si la API respondiera, por ejemplo, "usuario no existe" o "cuenta deshabilitada", un atacante podria probar listas de usernames, identificar cuales pertenecen a usuarios reales y concentrar ataques de fuerza bruta o ingenieria social sobre ellos. En PharmaBackend la autenticacion devuelve un 401 generico, mientras las validaciones internas distinguen el motivo sin exponerlo al cliente. Esta decision reduce la informacion disponible para un atacante y mantiene una respuesta uniforme frente a fallos de autenticacion.

2. Token anterior despues de cambiar la contrasena

Despues de cambiar la contrasena, un JWT ya emitido puede seguir siendo valido hasta su expiracion porque la API es stateless: el servidor no guarda una sesion individual que pueda cerrar y la firma del token continua siendo valida mientras no cambien el secreto ni la fecha de expiracion. En este proyecto, JwtAuthenticationFilter obtiene el username del token y vuelve a cargar el usuario actual desde la base de datos, pero el cambio de password por si solo no altera la validez criptografica del JWT. Una solucion seria agregar al usuario un campo tokenVersion o passwordChangedAt e incluir esa version/fecha como claim; el filtro compararia el valor del token con la base de datos y rechazaria tokens emitidos antes del cambio. Otra alternativa es usar una lista de revocacion con identificadores jti, especialmente para eventos de seguridad.

3. Exposicion del secreto JWT

Si el secreto JWT se publicara en GitHub, un atacante que lo conozca podria fabricar tokens con una firma valida. En esta implementacion, aunque las authorities efectivas se recargan desde la base de datos, un token falsificado con sub=admin permitiria intentar autenticarse como el usuario administrador y obtener sus permisos actuales. La respuesta inmediata debe ser rotar el secreto, reiniciar la aplicacion para invalidar todos los tokens firmados con la clave comprometida, retirar el secreto del repositorio y de su historial, y almacenarlo fuera del codigo mediante variables de entorno o un gestor de secretos. Tambien conviene revisar logs y actividad reciente para detectar posible uso indebido.

4. @PreAuthorize frente a reglas por URL

Proteger los reportes con @PreAuthorize en el controlador tiene la ventaja de colocar la regla de autorizacion junto al metodo que ejecuta la operacion, permitiendo expresar permisos especificos y conservando la proteccion aunque el metodo sea invocado mediante distintas rutas o cambie la configuracion global. Esto mejora la granularidad y hace explicita la regla de negocio en el punto de ejecucion. Como desventaja, las reglas pueden quedar distribuidas entre varios controladores y resultar menos visibles que una politica centralizada en SecurityConfig; ademas, es necesario mantener habilitada la seguridad a nivel de metodo y revisar que ningun endpoint sensible quede sin anotacion. Una combinacion de reglas generales por URL y @PreAuthorize para operaciones criticas ofrece defensa en profundidad.

Conclusion

Las pruebas realizadas confirman que PharmaBackend aplica correctamente autenticacion JWT, validacion de expiracion y firma, control de permisos por rol, restricciones administrativas, gestion segura de contrasenas, activacion/desactivacion de usuarios y CORS para la SPA. Los 28 casos documentados finalizaron en estado Pasa. La principal recomendacion de fortalecimiento es incorporar un mecanismo explicito de revocacion o versionado de tokens para invalidar JWT ya emitidos despues de un cambio de contrasena u otro evento de seguridad.

Sesion 10 - Actividad Autonoma